مقارنة أوزان النموذج قبل وبعد التدريب

قبل التدريب: ALLaM-7B-Instruct من سدايا (SDAIA) | بعد التدريب: آخر تدريب لنا (إسناد v5) | القياس: كل الأوزان في ٢٢٤ طبقة مدرَّبة

الخلاصة

- **التدريب غيّر `0.57%` فقط من النموذج.** درّبنا 40.0 مليون رقم من أصل 7.00 مليار. الباقي (`99.43%`) هو نفس أوزان سدايا حرفيًا، بدون أي تغيير.
- **الـ Bias: ما فيه أي bias في النموذج أصلًا، لا قبل ولا بعد.** ALLaM بنية Llama بدون bias (`attention_bias=false`)، وفحصت كل أرقام النموذج (٢٩١ مصفوفة): صفر مصفوفات bias. وإعداد التدريب `bias="none"`، وملف التدريب (٤٤٨ مصفوفة) ما فيه أي bias. فالفرق في الـ bias = صفر. وأوزان التطبيع (RMSNorm) ما اتغيّرت (مجمّدة).
- **حجم التغيير في الأوزان:** متوسط `1.06%` من حجم الوزن نفسه (أصغر تغيير `0.47%` وأكبر `2.69%`). تغيير صغير جدًا، لكنه كافٍ يغيّر سلوك النموذج في المهمة اللي دربناه عليها.
- **وين صار التغيير؟** الطبقات العليا (٢٤–٣١) تغيّرت أكثر (`1.24%`) من الطبقات الأولى (٠–٧: `0.89%`). وأكثر نوع تغيّر هو `gate_proj` (`1.85%`)، وأقله `down_proj` (`0.66%`).
- **التغيير يتركز في اتجاهات قليلة:** في `gate_proj` حوالي `80%` من التغيير في اتجاه واحد فقط، مع إن التدريب سمح بـ ١٦ اتجاه. معناه ممكن نجرّب رتبة (rank) أصغر مستقبلًا بدون خسارة كبيرة (يحتاج تجربة تأكيد).

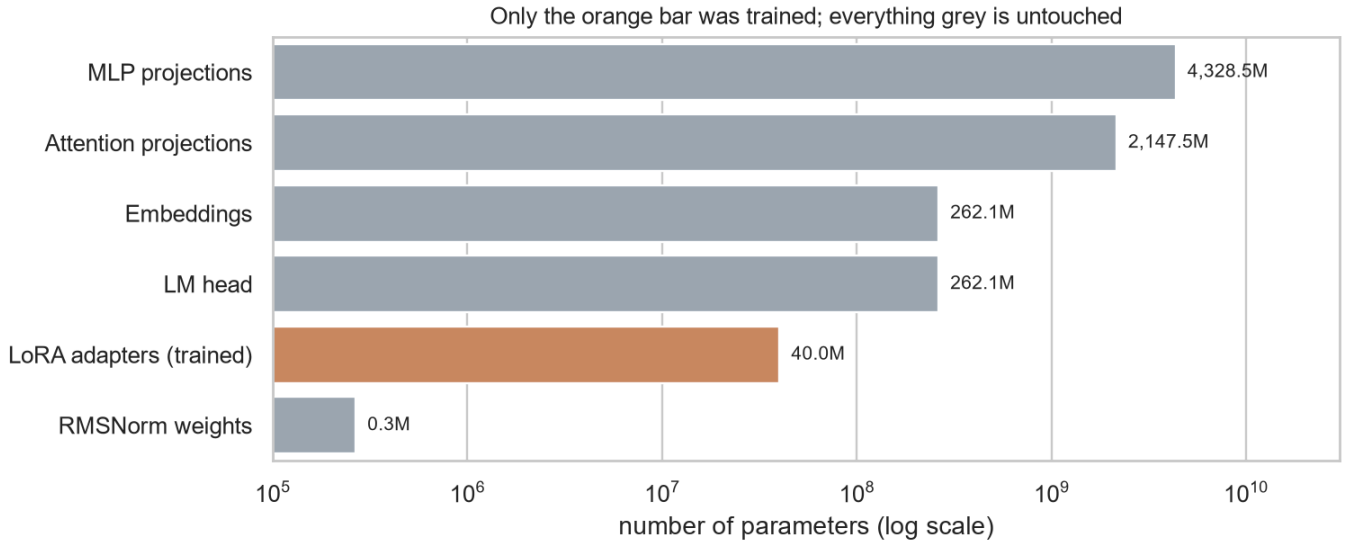
١. ماذا قارنّا بالضبط؟

	قبل التدريب	بعد التدريب
النموذج	`ALLaM-7B-Instruct (SDAIA)`	نفسه + LoRA adapter (إسناد v5)
الأوزان	7,000,559,616 رقم (bf16)	نفس الأوزان + تعديل على 39,976,960 رقم إضافي
الأجزاء اللي تتدرب	-	٧ مصفوفات في كل طبقة × ٣٢ طبقة = ٢٢٤ مصفوفة: `q, k, v, o, gate, up, down`
الـ Bias	لا يوجد	لا يوجد (`bias="none"`)
إعدادات التدريب	-	r=16, alpha=32, dropout=0.05, lr=0.0001, 1 epoch, effective batch 16, 4,416 train examples (276 steps), 4-bit QLoRA

كيف نحسب "الأوزان بعد التدريب"؟ LoRA ما يعدّل الوزن مباشرة، يضيف عليه تعديلًا صغيرًا:

$$W_{after} = W_{before} + (alpha / r) \cdot B \cdot A \quad (alpha / r = 2)$$

فالفرق بين قبل وبعد هو بالضبط $\Delta W = (alpha/r) \cdot B \cdot A$، وهذا اللي قسناه لكل مصفوفة. قبل التدريب B تساوي صفر، فالنموذج قبل التدريب = نموذج سدايا تمامًا.



الشكل ١: ميزانية الأرقام. اللي تدرّب هو الأعمدة البرتقالية فقط (40.0M)، وكل الباقي لم يُمس.

٢. الـ Bias

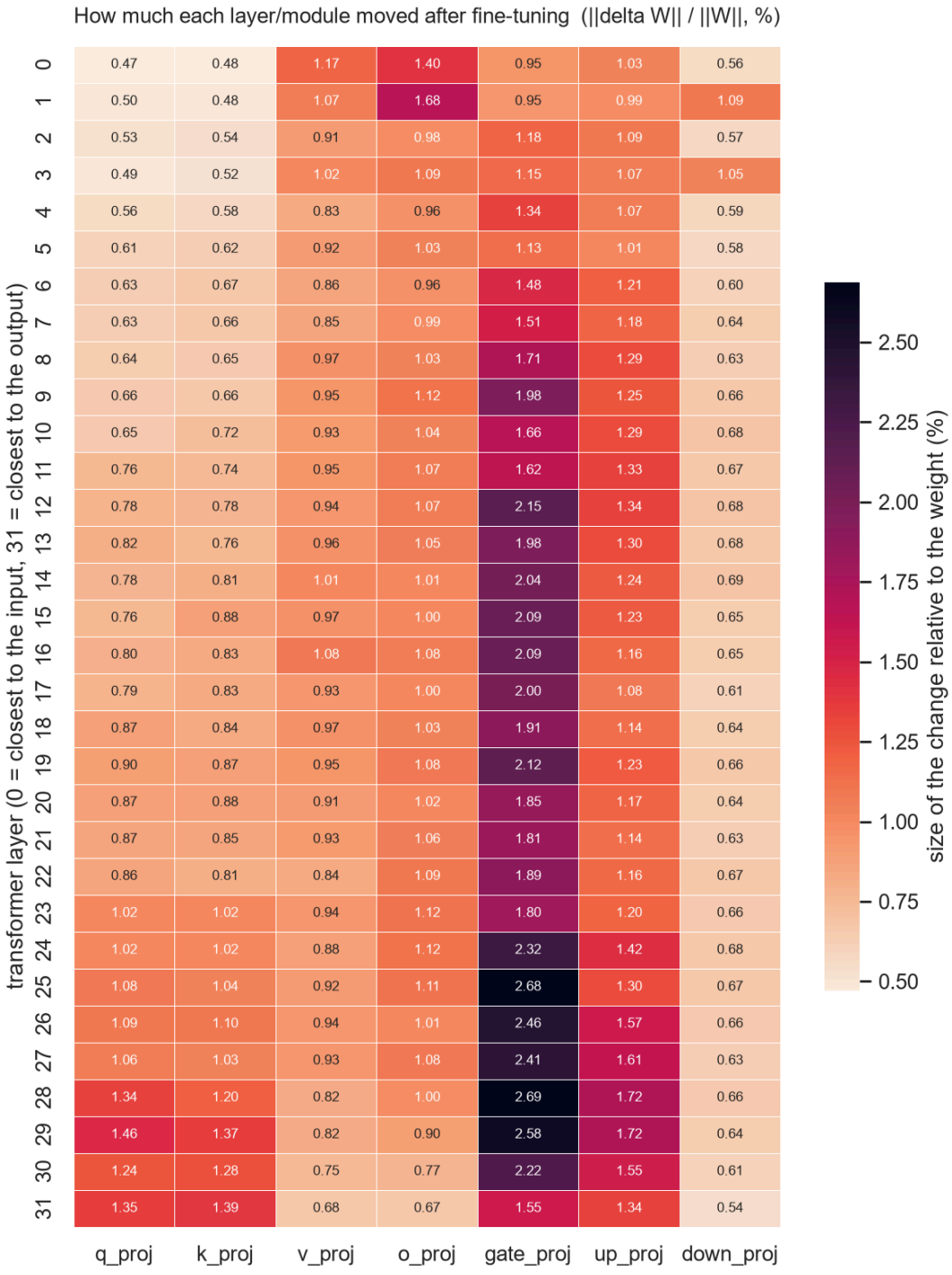
- الـ bias رقم إضافي يُضاف بعد كل ضرب مصفوفات ($y = Wx + b$). نماذج Llama، ومنها ALLaM، تعمل بدونه.
- فحص أسماء كل المصفوفات في ملفات سدايا الثلاثة: **٢٩١ مصفوفة، صفر منها اسمه ينتهي بـ `bias`**.
- فحص ملف التدريب (`adapter_model.safetensors`): ٤٤٨ مصفوفة (A و B لـ ٢٢٤ طبقة)، **صفر منها bias**.
- أوزان RMSNorm (266,240 رقم) كانت مجمّدة، فما اتغيّرت أيضًا.
- الخلاصة: **الفرق في الـ bias قبل وبعد = صفر**، وكل التغيير صار في الأوزان.

لو تقصد بـ "bias" الانحياز في المخرجات (مثل تكرار نفس المهام أو الأسماء)، هذا يُقاس بتجربة النموذج على أمثلة، مو من الأوزان. هذا موجود في تقارير الاختبار على أمثلة جديدة (`heldout_generalization_report_v5.md`). أقدر أجمعه في ملف مقارنة لو تبي.

٣. الأوزان: وين وكم تغيّرت؟

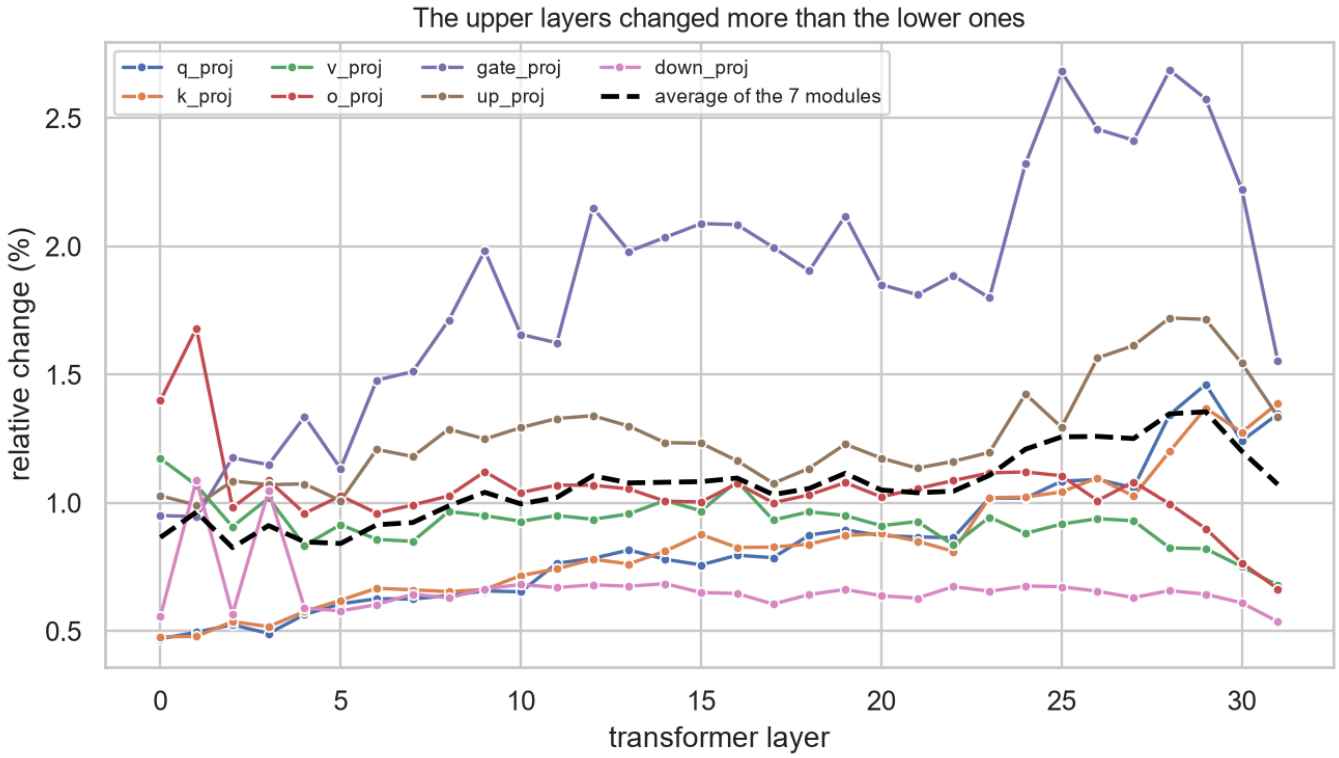
٣.١ خريطة حرارية (heatmap): كل طبقة وكل مصفوفة

كل خانة = مصفوفة وحدة. اللون الأغمق = تغيّرت أكثر نسبةً لحجمها. القراءة السريعة: عمود gate_proj (الخامس من اليسار) أغمق من البقية، والصفوف السفلية (الطبقات العليا) أغمق من العلوية.



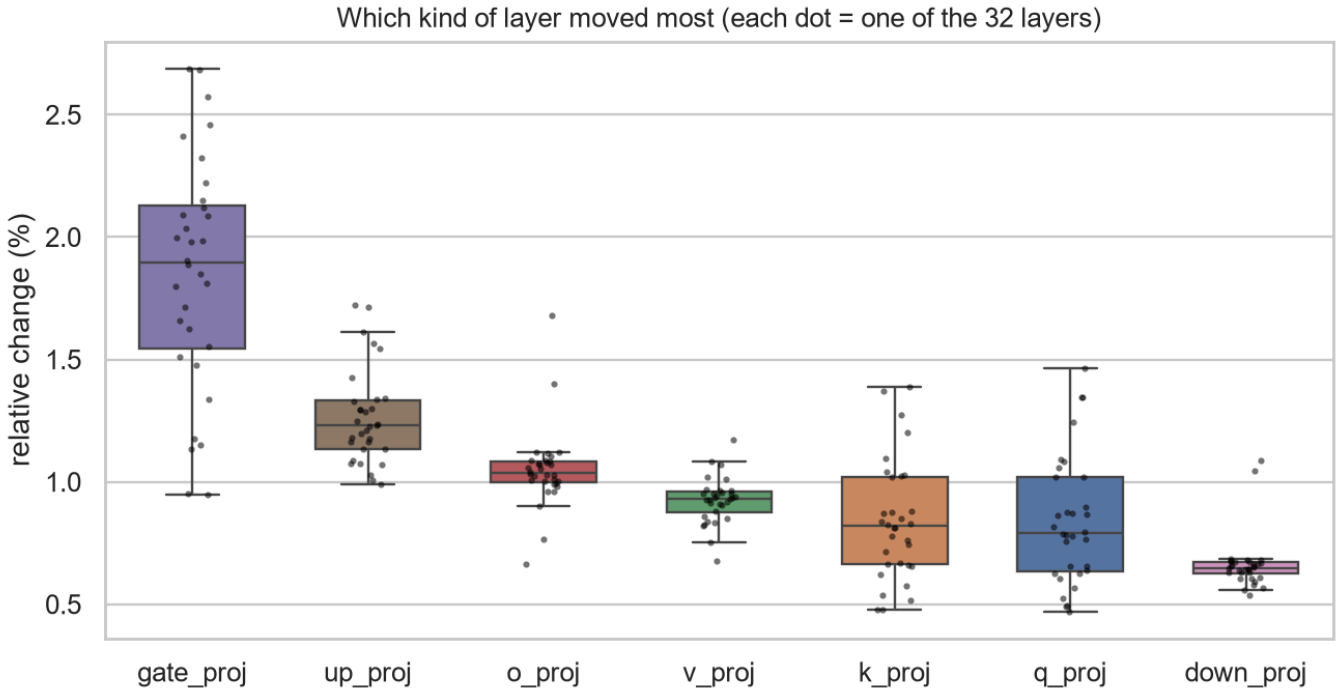
الشكل ٢: نسبة تغيّر كل مصفوفة (‖ΔW‖ / ‖W‖، بالمئة).

٣.٢ التغيير حسب العمق



الشكل ٣: التغيير يزيد تدريجيًا كل ما اقتربنا من مخرجات النموذج (الطبقات العليا)، وغالبًا لأن الطبقات القريبة من المخرجات مرتبطة أكثر بصياغة الإجابة (تفسير شائع في هذا النوع من التدريب، ما اختبرناه هنا).

٣.٣ حسب نوع المصفوفة



الشكل ٤: توزيع نسبة التغيير لكل نوع (كل نقطة = طبقة). متوسط الانتباه (attention) 0.91% مقابل الشبكة الداخلية (MLP) 1.26%.

النوع	متوسط التغيير	طاقة أهم اتجاه	الرتبة الفعلية (من ١٦)	أرقام تغيّرت بعد الدمج في bf16	تطابق الاتجاه بعد الدمج
q_proj	0.84%	51%	12.8	76%	0.978
k_proj	0.84%	49%	13.1	76%	0.978
v_proj	0.92%	46%	12.4	78%	0.984
o_proj	1.05%	30%	14.2	82%	0.987
gate_proj	1.85%	80%	9.9	86%	0.995
up_proj	1.26%	58%	12.6	83%	0.991
down_proj	0.66%	35%	13.8	72%	0.970

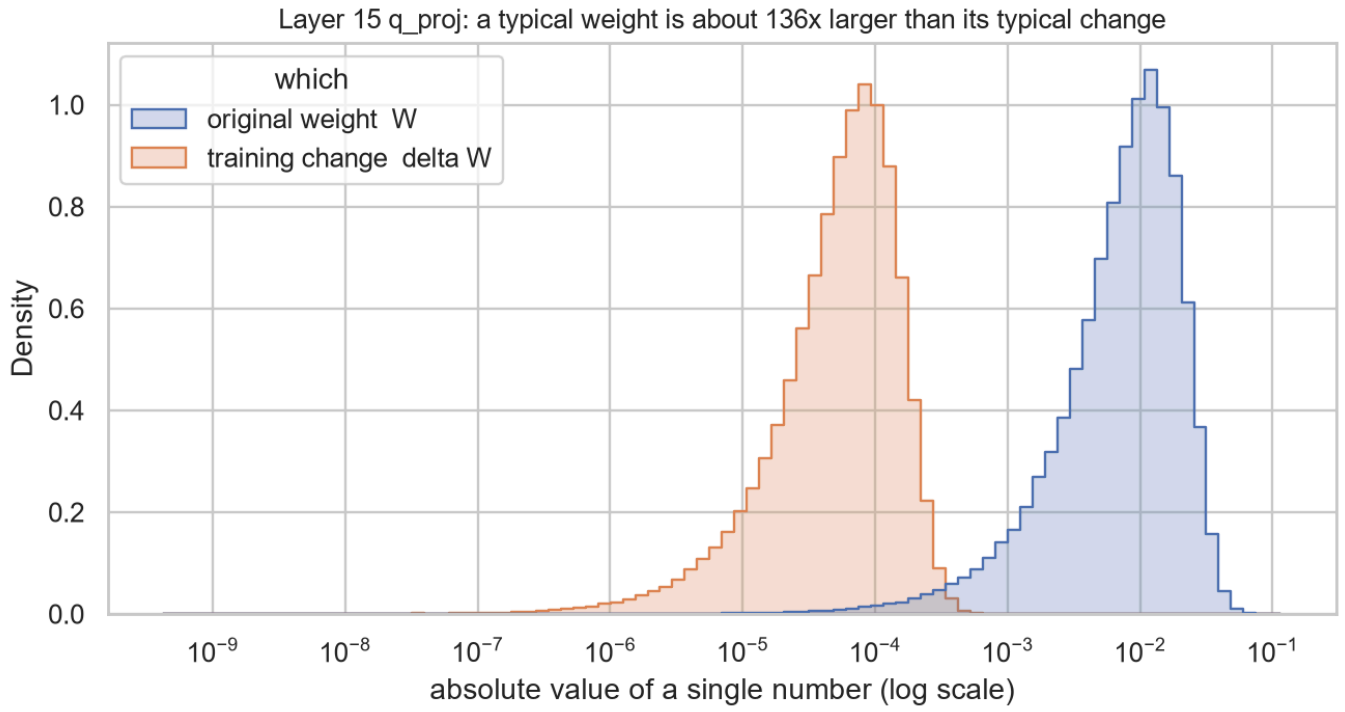
أكثر ٥ مصفوفات تغيّرًا

layer 28 · gate_proj	2.69%
layer 25 · gate_proj	2.68%
layer 29 · gate_proj	2.58%
layer 26 · gate_proj	2.46%
layer 27 · gate_proj	2.41%

أقل ٥ مصفوفات تغيّرًا

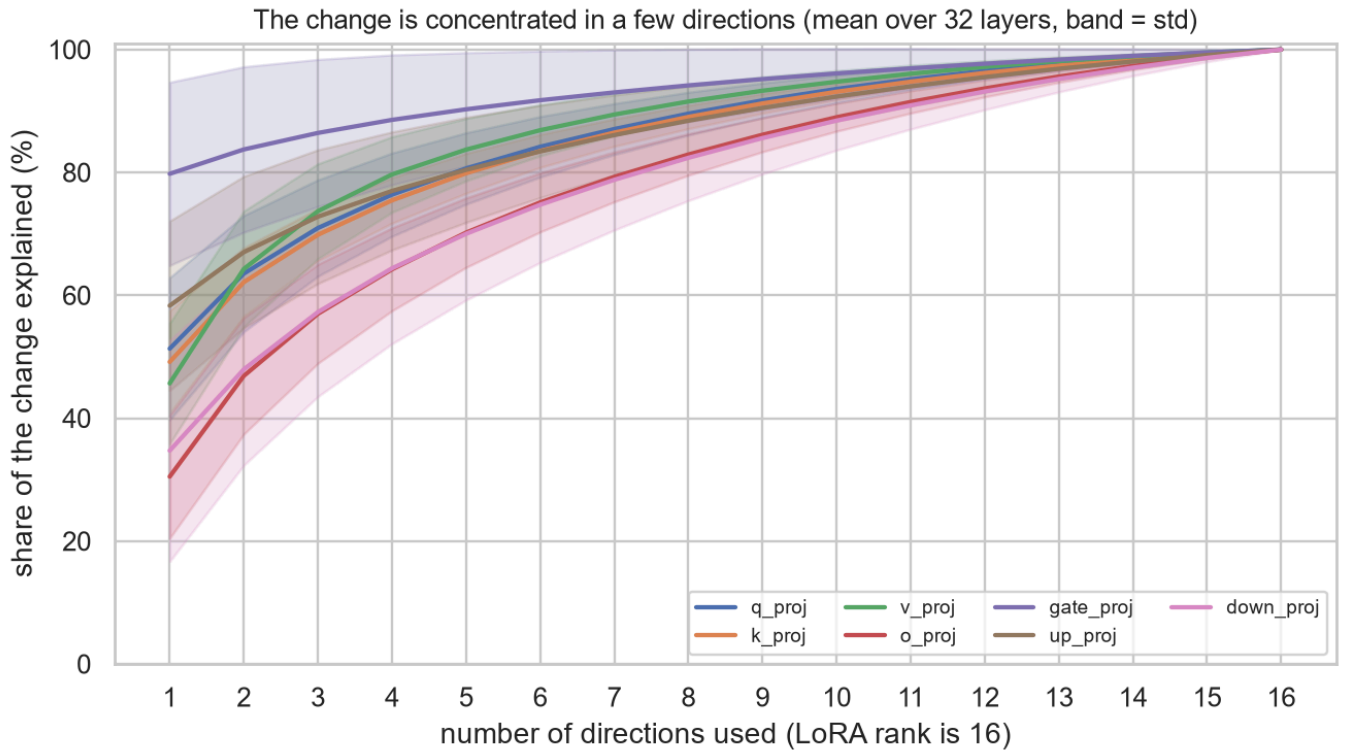
layer 0 · q_proj	0.47%
layer 0 · k_proj	0.48%
layer 1 · k_proj	0.48%
layer 3 · q_proj	0.49%
layer 1 · q_proj	0.50%

٣.٤ حجم التغيير مقابل حجم الوزن



الشكل ٥: التغيير النموذجي أصغر من الوزن النموذجي بمقدار **136 مرة** تقريبًا (مثال: طبقة ١٥، q_proj). يعني التدريب "هزّ" الأوزان هزّة خفيفة، ما غيّرها.

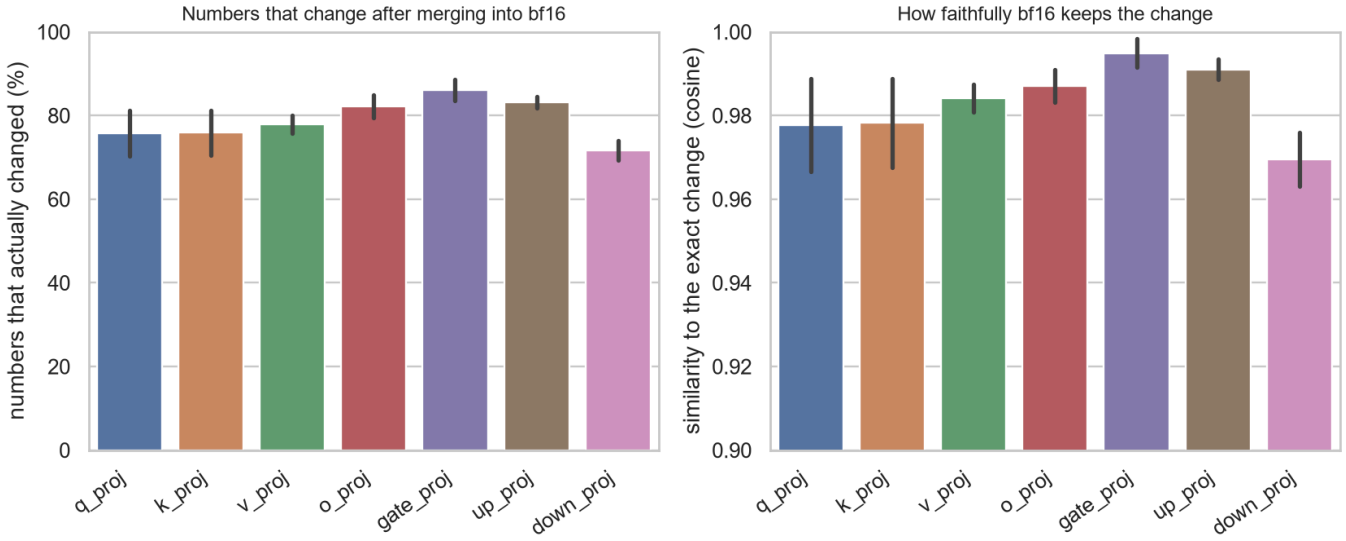
٣.٥ كم اتجاه استخدم التدريب؟



الشكل ٦: سمحنا بـ ١٦ اتجاه (rank). لكن في gate_proj اتجاه واحد يفسّر أغلب التغيير، وفي q/k_proj حوالي النصف. الاستنتاج: الرتبة ١٦ أكبر من المطلوب لبعض المصفوفات.

٤. ماذا يصير عند دمج التدريب داخل أوزان bf16؟

لو دمجنا التدريب في الأوزان لينتج نموذج واحد (بدل ملف منفصل)، الأرقام تنقرّب لدقة bf16 (حوالي ٢–٣ خانات عشرية). التغيير أصغر من الأرقام نفسها، فبعضه يضيع بالتقريب.



الشكل ٧: بعد الدمج، من 72% إلى 86% من الأرقام تتغير فعلًا. واتجاه التغيير الأصلي يبقى محفوظ بنسبة (cosine) من 0.97 إلى 0.99، يعني الدمج في bf16 **مقبول** وما يخرّب التدريب بشكل مؤثر.

ملاحظة: سيرفر الاستضافة يحمّل التدريب كملف منفصل فوق النموذج، فما يتأثر بهذا التقريب أصلًا. هذا القسم يهمنا فقط لو قررنا ندمج ونصدّر نموذج واحد.

٥. هل seaborn مفيدة هنا؟

نعم، وانستخدمت فعلًا في كل الرسوم. نصّبتها في بيئة منفصلة (`seaborn 0.13.2`) عشان ما نلمس بيئة التدريب.

- **الخريطة الحرارية (`heatmap`)**: أنفع استخدام. ٢٢٤ رقم (٣٢ طبقة × ٧ أنواع) ما تنقرا في جدول، لكن بالألوان تشوف النمط في ثانية. عملناها بسطر واحد.
- **الصندوق مع النقاط (`boxplot + stripplot`)** والخط مع الشريط (`lineplot` مع الانحراف المعياري): seaborn تحسبهم وحدها من الجدول، بدون كود إحصاء يدوي.
- **المدرّج التكراري (`histplot`)** بمقياس لوغاريتمي: خلّى مقارنة حجم الوزن بحجم التغيير ممكنة على رسم واحد.
- ما احتجناها لـ: الحساب نفسه (يتم بـ `torch`)، ولا للصور المعقدة. هي فوق `matplotlib`، فنفس النتيجة ممكنة بدونها لكن بكود أطول وألوان أسوأ.

٦. حدود هذي المقارنة

- هذا يقيس **كم** و**وين** تغيّرت الأوزان، مو **جودة** النموذج. الجودة تُقاس باختبارات الأمثلة الجديدة (تقارير `heldout`).
- التدريب والتشغيل على Modal يستخدمون نسخة مضغوطة بـ ٤-bit (QLoRA) من نموذج سدايا ويضيفون التدريب فوقها، لكن المقارنة هنا مع أوزان سدايا الأصلية بدقة bf16. فالأوزان اللي تشتغل فعليًا تختلف عنها قليلًا بسبب الضغط، وهذا الفرق مو من التدريب.
- النسبة المئوية صغيرة لأن LoRA يحدّث جزء صغير جدًا، فلا تعني إن النموذج "تغيّر قليلًا" في السلوك. أثر التدريب على المخرجات يُحكم عليه من الاختبارات.
- قارنّا آخر تدريب (v5) فقط مع نموذج سدايا. v4 وبقية المحاولات ما دخلت هنا.

٧. إعادة التشغيل

```
weights_stage1.py  (project venv: torch + safetensors) → per_module.csv, singular_values.csv, summary.json
weights_stage2.py  (plot venv: seaborn + matplotlib + pandas) → figs/*.png + this report
Total run time: about 20 seconds for stage 1 on an RTX 4070 Ti.
```